

CS4225/5425 Big Data Systems, Data Science

AY24/25 Sem 2. github.com/gerteck  
(Big) Data science. Interdisciplinary field about processes and systems to extract knowledge or insights from data in various forms.

- **Processing** diff. types of data (large data volume, graph data, stream data).
- **Models of Computation:** MapReduce/Spark, Large graph processing engines, Streams and online algorithms.

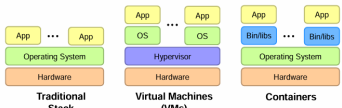
1. Introduction

- **Big Data (4 ‘V’s):** *Volume:* (Scale of data). *Velocity:* (Speed of new (streaming) data). *Variety:* (Different types). *Veracity:* (Accuracy, trustworthiness of data).
- **Challenges of 4 ‘V’s:** Performance, Cost, Reliability, Algorithm design complexity. Data integration challenges, multi-modal learning (Variety). Dirty and noisy data, data provenance, uncertainty (Veracity).
- Big data - Virtuous Product Cycle, where useful service → data to analyze → insightful action → useful service.

Utility Computing (Cloud Computing Infrastructure)

Cloud computing - scalable, elastic infrastructure for big data systems.

- **Utility Computing:** Computing resources as a metered service (“pay as you go”). Ability to dynamically provision virtual machines.
- **Scalable:** (“Infinite” capacity). **Elastic:** (Scale up or down on demand).



- **How: Virtual Machines (VMs):** Enable sharing of hardware resources, run each app in isolated virtual machine. High overhead as each VM has its own OS. **Containers:** Lightweight sharing of resources. Apps run in isolation but share same OS. Container encapsulates an app and environment.
- **Everything as a Service:**
  - *Infrastructure (IaaS):* User rents a virtual machine. (e.g. Amazon EC2).
  - *Platform (PaaS):* Provide hosting web apps. Cover hardware maintain, upgrade.
  - *Software (SaaS):* Pre-built apps. (e.g., Gmail). Covers entire stack.

Data science, cross-disciplinary, emerging research area w. applications in engineering, commerce, etc. Clouds as natural infrastructures for data science.

2. Principles of Big Data Systems

- **Data Center Infrastructure**
  - **Single Server:** A commodity hardware unit.
  - **Rack:** Dozens of servers, connect to rack switch (top-of-rack switch).
  - **Data Center:** Hundreds of racks, connect to data center switch (core switch).
- **Bandwidth vs. Latency**
  - **Bandwidth:** Max amt of data transmittable per unit time (e.g., in GB/s). Calculate as minimum bandwidth among all components in the data access path.
  - **Latency:** Time taken one packet travel src to dest (one-way) / round-trip (in ms). Sum of the latency caused by all components in the data access path.
  - **Throughput:** Rate data is actually transmitted in network in given time period.
- When transmitting (*large/small*) amount of data, (*bandwidth/latency*) - roughly how long transmission will take (bottleneck variable).
- **Storage Hierarchy in Data Centers:** based on capacity, latency, bandwidth.
  - **DRAM:** Fast but limited capacity. **Flash:** Intermediate btw. DRAM and disk.
  - **Disk:** Slow but large capacity. Disk reads more expensive than DRAM (higher latency, lower bandwidth).

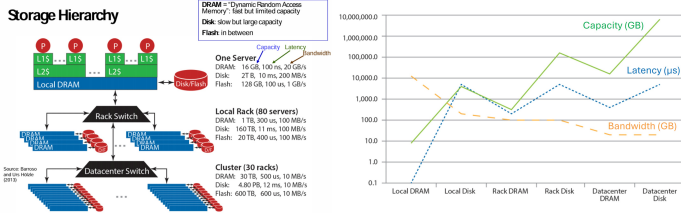
Data Flow Paths

Data flow paths depend on the location of the source and destination. Data access may cross several components, e.g. hard disk, DRAM, rack switch, cluster switch.

- E.g. Local DRAM → Local Disk in same server.
- Server A DRAM to server B DRAM in same rack (i.e. Local DRAM → Rack Switch → Server B’s DRAM)
- Transfer across different rack, same cluster etc. (i.e. Local DRAM → Rack Switch → Datacenter Switch → Server C’s Rack Switch → Server C’s DRAM).
- To load from disk, usually need to load into DRAM first. (i.e. Local Disk → Local DRAM → Rack Switch etc.)

Calculate latency, bandwidth based on data flow path. Approximate order of magnitude, for simplify, assume one-way communication, no failures.

Key metrics: different levels of storage and memory systems



Capacity increases, Costs (Latency increase, bandwidth decrease) increases from local to rack to data center, along storage hierarchy.

Building Efficient Big Data Systems

1. **Scale “Out”, Not “Up”:** Horizontal Scaling over vertical scaling. Combine many cheaper machines (out) over increase power of individual machine (up).
2. **Seamless Scalability:** Ideal (linear) scalability, performance increases proportionally to added resources.
3. **Move Processing to the Data:** As clusters limited bandwidth; move tasks to machine where data is stored.
4. **Process Data Sequentially, Avoid Random Access:** Disk seeks expensive; sequential access provides better throughput.

Challenges in Big Data Systems

1. **Machine Failures:** With enough scale, expect daily server machine failure.
2. **Synchronization:** Difficult due to unknown order of worker execution, interruptions, and shared data access. Tools like semaphores, conditional variables, and barriers are used to address synchronization issues.
3. **Programming Difficulty:** Concurrency hard to reason about, especially at scale. Debugging and managing failures add complexity.

API for Data Centers: Given challenges, “API” of data center abstracts system-level details from developers. Execution frameworks (e.g., Hadoop, Spark) handle task assignment, replication, synchronization, etc.

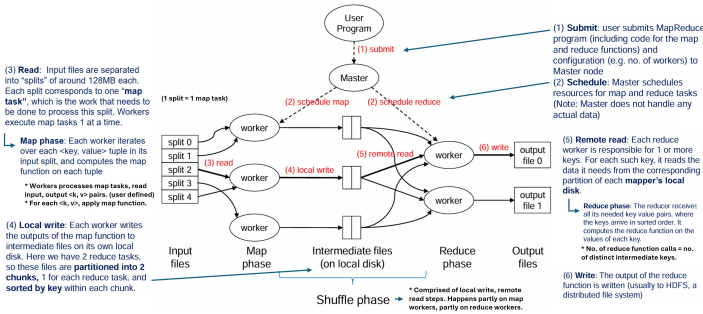
3. MapReduce

Big data requires new prog. abstractions, runtime systems. Hadoop MapReduce widely used for efficient, process large datasets in e.g. databases, data mining. **MapReduce:** prog. model & associated runtime system to process large datasets in parallel across distributed cluster. (Parallel + Divide & Conquer Paradigm). MapReduce provides functional abstractions for two operations.

- **Map function:** (k1, v1) → List(k2, v2). Processes each input key-value pair and emits intermediate key-value pairs.
- **Reduce:** Aggregate intermediate results to generate the final output. reduce(k2, List(v2)) → List(k3, v3). All values with same key sent to same reducer.

• Typical Workflow of MapReduce:

Iterate over a large number of records. Extract relevant info from each. (**Map**) Shuffle and sort intermediate results by key (**Shuffle**). Aggregate intermediate results (**Reduce**). Generate the final output.



- **Worker:** single component of cluster, performs storage, processing tasks.
- **Map Task:** A unit of work, processing one input split (e.g., 128 MB).
- **Map Function:** A call to user-defined map function, for each input split k-v pair.
- **Reduce Task:** A unit of work processing a subset of intermediate keys.
- **Barrier:** barrier between map and reduce phases. Else, reduce phase may compute wrong answer. Shuffle phase can begin copying intermediate data earlier.

Partition and Combiner

Optionally specify **partition, combine** functions, optimize, reduce network traffic.

- **Partition Step:** By default, keys assigned to reducers using hash function (e.g., hash(key) % num\_reducers). Can implement custom partitioner to balance load among reducers. (E.g. if some keys much more values than others).
- Partitioner runs in local write phase, knowledge of which keys to which reducer.
- **Combiner Step:** Combiners locally aggregate map outputs to reduce disk and network I/O. (‘mini-reducers’) (E.g. Sum counts for each key before write to disk). *Correctness:* Must not affect correctness of final output, regardless of how many times it runs (includ. 0 times). E.g. sum, max ok, but mean, minus not ok.
- Combiner runs after map phase, done during local write phase, before actual disk write to save disk I/O.

Notes and Other Optimizations

- If reduce task handles multiple keys, will process keys in sorted order.
- **Secondary Sort:** Use this property to sort values associated with key. (reduce work done in reducer by using inbuilt distributed sorting process of Hadoop). Combine key and value into composite key e.g. (UserID → [UserID-Timestamp]), define custom comparator (sort by ID then timestamp) & partitioner (group by ID only).
- Size of split too big or small: → Limited parallelism vs. high overhead.
- #map fn calls = # input key-value pairs. # map tasks = # input splits.
- Reduce function calls = no. of distinct intermediate keys.
- **Preserving State in Map / Reduce Tasks:** In MapReduce, state variables can be used to store information that persists across multiple calls to map() or reduce() functions within same task. Sharing data or resources between function calls.

General Performance Guidelines (Basic Algorithm Design)

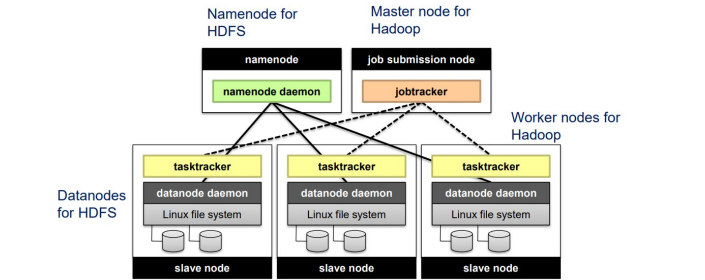
- **Linear Scalability:** More nodes should enable more work done in same time.
- **Minimize Disk and Network I/O:** Sequential access over random access. Send data in bulk over small chunks.
- **Reduce Memory Working Set:** Avoid large in-memory data structures to prevent out-of-memory errors.

4. MapReduce & Databases

HDFS: Hadoop Distributed File System

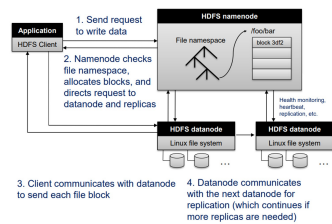
Distributed file system designed to store, manage large datasets across multiple machines in a cluster. Core component of Apache Hadoop ecosystem (open source), optimized for high-throughput access to big data.

- **Purpose:** handle massive datasets (tb/pb), scalable, fault-tolerant storage.
- E.g. MapReduce input, output stored in HDFS (3 copies etc.) Intermediate files on hard disk on local machine though (local fs).
- **Architecture: Master-Slave Model.**  
**NameNode:** Master node, manage file system metadata (e.g., dir structure, file locations, rebalance, heartbeat) - No data moved through NN. If NN data lost, cannot reconstruct files from raw block data. Hadoop has NN backup, checkpointing.  
**DataNodes:** Worker nodes, store the actual data blocks.



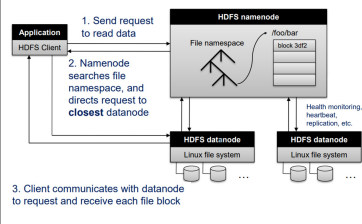
- **Data Locality:** Move computation to data, over other way round. Minimize network traffic, improve performance. Same set of nodes used for both HDFS, Hadoop. Hadoop tries to schedule map tasks run on machines containing data.
- **Fault Tolerance:** Each block replicated (default: 3 copy) across different DataNodes to ensure reliability. If node fails, HDFS retrieves data from another replica.
- **Rack Awareness:** Aware of physical location of nodes (e.g., which rack). Replicas placed on diff rack, balance fault tolerance, network efficiency.
- **Block Size:** Files divide to large chunks (default: 128 or 256 MB), reduce overhead. Large block sizes improve sequential read/write perform. (↑ throughput).
- **Scalability:** Scales horizontally by adding more machines to the cluster. Supports thousands of nodes and handles large datasets efficiently.
- **Write Once, Read Many:** Files written once, can be read multiple times. Files cannot be modified after written (append-only in some cases).
- **Use Cases:** Store log files, web server data, social media data, etc. Support big data processing frameworks like MapReduce, Spark, Hive, and Pig.

HDFS Architecture: Writing Data



**Disadvantages of HDFS:** Not suited for small files, inefficient for large no. of small files, due to metadata overhead. High latency, not ideal for real-time or low-latency applications. Limited Random Access: design for seq access, not random reads/writes.

HDFS Architecture: Reading Data

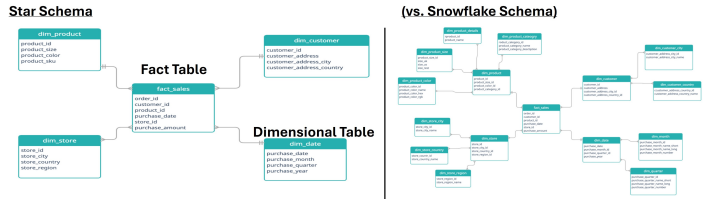


Relational Databases DB

Relational database comprised of tables, each table represents a relation, which is a collection of tuples (rows). Each tuple consists of multiple fields. Develop large-scale relational database using Hadoop / MapReduce.

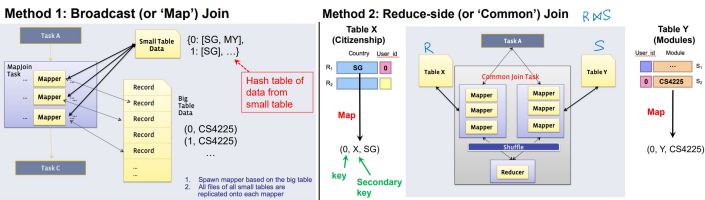
Star Schema and SQL Queries

The Star Schema is a common database schema for data warehousing. Organizes data into single (central) fact table and dimension tables. Fact table linked via foreign key r/s to PK of each dimension. Faster query and simpler queries.



SQL (Relational) Operations in MapReduce

- **Projection in SQL/MapReduce:** Projection selects specific columns from table. (E.g. SELECT <columns> FROM Sales). Only selected columns returned.
  - **Map Phase:** Input: Tuples (t. ID as key), emit tuples (w. only select attributes).
  - **No Reducer Needed:** "map-only" job, no need for shuffling or reducing.
- **Selection in SQL/MapReduce:** Selection filters rows based on condition. (E.g. SELECT \* FROM Sales WHERE price > 10). Only rows satisfying the predicate (condition) are returned. MapReduce implementation:
  - **Map Phase:** Input: Tuples (tuple ID as key) emit tuples satisfying predicate.
  - **No Reducer Needed:** Similar to projection, this is a map-only job.
- **Group By and Aggregation:** Aggregation operations. Shuffle phase is the 'group by'. Framework auto groups tuples by key. (E.g. SELECT product\_id, AVG(price) FROM sales GROUP BY product\_id). Output avg per product.
  - **Map Phase:** Emit key-value pairs: (product\_id, price).
  - **Shuffle Phase:** Framework groups tuples by key (product\_id).
  - **Reduce Phase:** Compute average price for each product\_id. (Same Reducer). Optimize using combiners to reduce network traffic.
- **Relational Joins (Inner Join):** Joins combine data from two tables based on a common key. (E.g. SELECT A.Id, A.attr1, B.attr2 FROM A JOIN B ON A.bID = B.bID) Many-to-many r/s (can appear multiple times in both tables).
  1. **Broadcast (Map) Join:** Requires the smaller table to fit in memory. Mappers store copy of small table as hash table. Iterate over large table, perform join. (Hash Join. Smaller table 'broadcasted' to all mappers/processing nodes.)
  2. **Reduce-side (Common) Join:** Mappers emit records with join key as primary key. Reducers group records by join key. Can use secondary sorting to ensure all keys from one table arrive before the other. Hold keys from first table in memory, cross them with records from second table (to do join).



- Big data systems do not automatically guarantee performance benefits.
- Performance analysis, algorithm design crucial for efficient, effective applications.

5. MapReduce & Data Mining

Similarity Search

Similarity search is a fundamental problem in data mining and machine learning, often used to find objects "similar" to each other. Real-world applications e.g.:

- Finding documents with similar words for duplicate detection, topic classification. Clustering similar news articles, research papers.
- Identifying customers purchasing similar products for recommendation systems.
- Grouping users with similar browsing behaviors for targeted advertising.

Distance and Similarity Measures

Between objects  $x$  and  $y$ , use **distance measure**  $d(x, y)$  or **similarity measure** (opposite). 'Near Neighbours' are points small distance apart. Common metrics:

- **Euclidean Distance:** Straight-line distance btw two points in space.
- **Manhattan Distance:** Sum of absolute differences along axes.
- **Cosine Similarity:** Measures the cosine of the angle between two vectors, focusing only on direction (ignoring magnitude).
- **Jaccard Similarity:** Similarity between sets  $A, B$ :  $s_{Jaccard}(A, B) = \frac{|A \cap B|}{|A \cup B|}$
- **Jaccard Distance:** Complement  $s_{Jaccard}$ :  $d_{Jaccard}(A, B) = 1 - s_{Jaccard}(A, B)$

Case Study: Finding Similar Documents

- Problem Definition:** Given a large collection of  $N$  documents:
- **All-Pairs Similarity:** Find all pairs of docs w. Jaccard similarity above threshold.
- **Similarity Search:** Given a query document, find all documents similar to it.

**Approach:** To efficiently find similar documents, we do shingling and min-hash.



Shingling

- A **k-shingle** (or k-gram) is sequence of  $k$  tokens (words / chars) that appears in document. E.g. 'the cat is glad' → set of 2-shingles: {'the cat', 'cat is', 'is glad'}
- **Similarity Metric for Shingles:** Each document  $D_i$  represented as set of shingles  $C_i$ . Similarity of  $(D_1, D_2)$ :  $s_{Jaccard}(D_1, D_2) = \frac{|C_1 \cap C_2|}{|C_1 \cup C_2|}$
- **Challenge:** For large datasets, comparing all pairs directly computationally expensive ( $O(N^2)$  comparison). To address this, use **Min-Hashing**.

Min-Hashing (Each shingle hashes to a int, get min int value of all shingles)

Convert large sets of shingles into short signatures while preserving similarity. Highly similar = high probability of same signature. Dissimilar docs low prob.

1. Define a hash function  $h$  that maps each shingle to an integer.
  2. For each document, compute min hash value across all shingles → 'signature'.
  3. Use *multiple* independent hash functions to generate *multiple* signatures.
- **Key Property of Min-Hashing:** probability two documents have same MinHash signature is equal to Jaccard similarity.
  - **Candidate Pairs:** Compare signatures (e.g.  $> k$  matching signatures). Pairs either directly used as output or further compared to confirm similarity.

MapReduce Implementation

- **Map Phase:** Read each document, extract shingles. Hash each shingle, compute MinHash signature. Emit (signature, document\_id). (Either run multiple times for diff hash, or use signature\_vector, custom partitioner to send same reducer.)
- **Reduce Phase** Receive all documents with the same MinHash signature. Generate candidate pairs from these documents. (Optional, confirm pair further similarity).
- **Parallelism:** # map node → # docs chunks. # reduce node → # keys (signature).



Clustering

- **Clustering:** Fundamental technique in data mining, machine learning to group unlabeled data points into clusters of similar items. Ideally, points have high intra-cluster similarity, low inter-cluster similarity.
- **Applications of Clustering:** Microbiology gene/protein identification, recommendation systems, social networks, information retrieval.

K-Means Algorithm

- Clustering technique. Separates data points into  $K$  clusters by iteratively optimizing positions of cluster centers.
1. **Initialization:** Randomly select  $K$  points as initial cluster centers.
  2. **Repeat until convergence:**
    - (a) **Assignment Step:** Assign each data point to nearest cluster center based on distance metric (e.g., Euclidean distance).
    - (b) **Update Step:** Move each cluster center to average (mean) position of all points assigned to it.
  3. **Stopping Condition:** Stop when no assignments change between iterations (i.e., the algorithm converges).

**Challenge with K-Means:** Iterative algorithms like K-Means are not inherently optimized for distributed systems like MapReduce!

MapReduce Implementation of K-Means

**MapReduce Implementation v1**

```
1: class MAPPER
2:   method CONFIGURE()
3:   c ← LoadCLUSTERS()
4:   method MAP(id i, point p)
5:   n ← NEARESTCLUSTERID(chusters c, point p)
6:   p ← EXTENDPOINT(point p)
7:   EMIT(cluster n, point p)
```

**Naive** – Disk I/O between mapper & reducer =  $O(n\ m\ d)$

**MapReduce Implementation v2 (with in-mapper combiner)**

```
1: class MAPPER
2:   method CONFIGURE()
3:   c ← LoadCLUSTERS()
4:   H ← INTAssociativeARRAY()
5:   method MAP(id i, point p)
6:   n ← NEARESTCLUSTERID(chusters c, point p)
7:   p ← EXTENDPOINT(point p)
8:   H[n] ← H[n] + p
9:   method CLOSE()
10:  for all cluster n ∈ H do
11:    EMIT(cluster n, point H[n])
```

**Optimize** – Disk I/O =  $O(k\ m\ d)$  (\* # map tasks)

**Reducer**

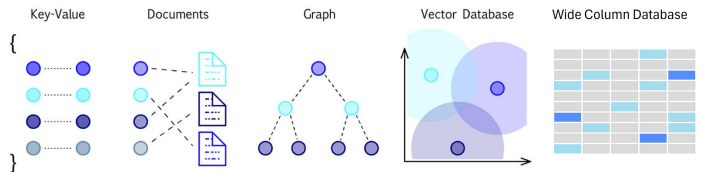
```
1: class REDUCER
2:   method REDUCE(cluster n, points {p1, p2, ...})
3:   s ← INTPointSUM()
4:   for all point p ∈ points do
5:     s ← s + p
6:   m ← COMPUTECENTROID(point s)
7:   EMIT(cluster n, centroid m)
```

**Annotations:**

- Network traffic!** (v1)
- Configure()** reads in the current positions of the cluster centers (i.e. stars) from a file.
- Map()** assigns each point to the closest cluster.
- Reduce()** aggregates the points in each cluster.
- In-mapper combiner used** (v2)
- High Memory Usage** (v2)

MapReduce Usage in Data Mining, Machine Learning

- MapReduce still needs to iteratively run till convergence.
- Iterative algorithms like K-Means maybe not ideal for MapReduce, high network, disk I/O overhead (input disk read for every iteration).
- **In-mapper combiner:** aggregate values directly in mapper using in-memory structures to emit fewer k-v pairs, reducing number of combiner calls. Reduce disk I/O since fewer intermediate records, but higher memory usage in mapper.
- **System Combiner:** Use dedicated combiner function (run after map phase before data sent to reducer), run by system. More suitable when memory is limited.
- **Secondary Sort** - No aggregation, records (same key) go reducer in specific order.
- While optimizations such as in-mapper combiners can reduce computational cost, but not fully address inefficiencies of iterative algorithms in MapReduce. (Other frameworks e.g. Spark can overcome instead by using caching, in memory)



NoSQL & Distributed Databases

- NoSQL: non-relational databases, store data in formats other than relational tables.
- “SQL” here refers to traditional relational DBMS (relation databases, not querying language). NoSQL systems often involve SQL-like querying languages.
  - “NoSQL” originally “Not SQL,” but evolved to mean “Not Only SQL”.
  - **Coexistence** of relational & relational databases in modern systems, each for tasks they are suited for, e.g. graph, vector databases.
  - **Key Motivations:** Growth in web apps, big data, distributed systems.
    - **Volume and Velocity:** Handling large-scale data and high-speed data ingestion.
    - **Scalability:** Horizontal scalability to distribute data across multiple servers.
    - **Flexibility:** Support for unstructured/ semi-structured data w. flexible schemas.
    - **Performance:** Optimize specific use case, e.g. fast read/write, efficient storage.

Major Types of NoSQL Databases

- NoSQL databases categorizable into several types, based on their data models:
1. **Key-Value Stores:** Store data as key-value pairs (e.g., Redis, DynamoDB).
  2. **Wide Column Databases:** Store data in tables with rows and columns, but allow flexible schemas (e.g. Cassandra, HBase).
  3. **Document Stores:** Store as documents, JSON-like (e.g. Mongo, CouchDB).
  4. **Graph Databases:** Data as nodes, edges to model relationships (e.g. Neo4j).
  5. **Vector Databases:** Store high-D vectors for similarity search (e.g., Milvus).

Key-Value Stores

- **Data Model:** Key-value stores associate keys with values.
- **Keys:** Unique identifiers. Primitives e.g. ints, strings, raw bytes, queryable.
- **Values:** can be simple or complex data types, e.g. primitives, lists, JSON objects, or binary large objects (BLOBs). Usually cannot be queried.
- **Operations:** simple API, Get (fetch value of key), Put (set value associated w. key). Optional operations include Multi-get, multi-put, range queries.
- **Use Cases:** Optimized for fast reads & writes, caching, session storage, small-continuous read/write, uses w. simple query patterns (e.g., Redis, DynamoDB).
- **Persistence: Non-persistent** (just big in-memory hashtable, but back up data to disk periodically) e.g. Memcached, Redis. **Persistent:** Data stored persistently to disk. E.g. RocksDB, Dynamo, Riak

Document Stores

- **Data Model:** Organize data into collections of documents.
  - **Database** contains multiple collections. A **collection** contains multiple documents. A **document** is a JSON-like object with fields and values.
  - Documents can be nested (JSON objects as values), varying fields, flexibility for unstructured / semi-structured data.
- **Querying:** Unlike basic k-v stores, document stores support querying based on document content. Common operations: CRUD (create, read, update, delete).
- **Example: MongoDB Architecture:** MongoDB uses a distributed architecture.

**Architecture of MongoDB**

**Replica sets:** multiple processes which maintain the same data, for redundancy

Each replica set is responsible for a subset of the database, known as a **shard** (this is a **horizontal partitioning** scheme)

**Example of Read or Write Query**

```
Find(class='cat201')
//name='bob' value='cat201'
//name='bob' value='cat201'
```

```
Insert(class='cat201')
//name='bob' value='cat201'
```

```
Find(class='cat201')
//name='bob' value='cat201'
```

```
Find(class='cat201')
//name='bob' value='cat201'
```

1. Query is issued to a router (mongos) instance
2. With config server, mongos determines which shards to query
3. Query sent to relevant shards (partition pruning)
4. Shards run query on their data, send results back to mongos
5. mongos merges the query results and returns the merged results to the application

**Replication in MongoDB**

- Standard configuration:
  - 1 primary
  - 2 secondaries
- Writes:
  - The 'Primary' receives all write operations
  - Records writes onto its 'operation log'
  - Secondaries will replicate operation log, apply to local copies of data (thus ensuring data is synchronized), then acknowledge operation
- Reads:
  - The user can configure 'read preference', whether can read from secondaries (secondary), or the primary only
  - Allowing reading from secondaries can decrease latency (improving throughput), but allows for reading stale data (thus only 'eventual consistency')

Wide Column Databases

- Store data in tables w. rows & columns, like relational DB, allow flexible schemas.
- Each row can have different columns, data partitioned by partition key column.
  - **Column Stores:** 'Mix between relational database and document store'.
  - Ideal for large-scale, distributed datasets (e.g., Cassandra, HBase, open source).

Graph Databases

- Represent data as nodes (entities) and edges (relationships), enabling efficient traversal and querying of relationships.
- These databases are particularly useful for applications like social networks, recommendation engines, and fraud detection, where relationships between entities are critical (e.g., Neo4j).

Vector Databases

- **Vector Databases:** Each row represent point in  $d$ -dimensional space.
- Vectors typically represent embeddings from machine learning models. These databases enable fast similarity searches, such as finding nearest neighbors, and are widely used in AI/ML workflows for tasks like recommendation systems, image search, and clustering (e.g., Milvus, Pinecone).
- Algorithms like Locality-Sensitive Hashing (LSH) enable fast similarity search.
- **Examples:** Milvus, Redis, Weaviate, MongoDB Atlas

Pros and Cons of NoSQL Systems

- **Advantages:** Flexible schemas for semi/unstructured data. Horizontally scalable. High performance and availability due to relaxed consistency models.
- **Disadvantages:** Lack of declarative query languages (e.g., SQL). Weaker consistency guarantees, which may require application-level handling. Challenges with schema evolution and data duplication.
- Choice between NoSQL and relational databases depends on: *app requirements* (e.g., consistency vs. availability), query complexity (e.g., joins vs. simple read/writes), data volume and scalability needs.

Key Concepts in NoSQL / Distributed Databases

- **Brewer's / CAP theorem:** Any distributed system/ data store can simultaneously provide only 2 of 3 guarantees: consistency, availability, partition tolerance (CAP).
- **BASE:** Basically Available, Soft state, and Eventually consistent.
  - **ACID** databases prioritize consistency (correctness) over availability—the whole transaction fails if an error occurs in any step within the transaction.
  - **BASE** databases prioritize availability over consistency. Basic read write available most of time. Instead of fail, users access inconsistent data temporarily (*soft state*, no guarantees). Data consistency is achieved eventually.
- **Consistency Models** Different from ACID consistency - data integrity.
  - **Strong Consistency:** Any reads after update give same result on all observers.
  - **Eventual Consistency:** Updates propagate over time, and readers may see stale data temporarily. Offers higher availability but weaker guarantees.
- **Denormalization:** (duplication of data across tables) - tradeoff between query efficiency and propagation of data updates. Design around queries expected.
- **Partitioning** across distributed databases: (e.g. by Range / specific column) - Related queries efficient, (entries co-located, no need scan entire database). However, may have imbalanced partitions, maintenance (shift from one partition to another on change), inefficiency of incompatible queries - need scan multiple partitions.

**NoSQL Overall:** Evaluate pros cons, select right DB system for app use case

- NoSQL DB good for modern apps, flexible dynamic schema, (horizontal) scalability, performance & availability (due to sacrificing strong consistency, removing need for locks, concurrency mechanisms).
- Trade-offs in consistency guarantees (outdated data, additional work to handle on app side), and query capabilities (e.g. no joins).